Modelos y servicios de datos
MYSD-MBDA

Integridad. Procedimental.

CEIS

2026-2

Agenda

- Integridad
  - Lo básico
  - Mecanismos
  - Caso: Envíos
- Acciones
  - De base de datos
  - De lenguaje imperativo
  - Excepciones
- Integridad procedimental
  - Acciones referenciales
  - Disparadores
- Caso: Ventas

Agenda

Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Agenda

Integridad

- Lo básico
- Mecanismos
- Caso: Envíos

Acciones

- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental

- Acciones referenciales
- Disparadores

Caso: Ventas

Modelo relacional

Aspectos basicos

- **Aspecto estructural**

  Los usuarios perciben la información de la base de datos como tablas y nada más que tablas

- **Aspecto de integridad**

  Estas tablas satisfacen ciertas restricciones de integridad

- **Aspecto de manipulación**

  Las operaciones disponibles permiten derivar tablas a partir de tablas

Integridad

El ideal: la regla de oro

Nunca debe permitirse una operación de actualización que deje cualquier tabla en un estado que viole su propio predicado.

Un mecanismo: aspecto de integridad

Hacer que las tablas cumplan condiciones de integridad

Agenda

Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Estrategias de integridad

Declarativa

¿QUÉ? Indicar qué debe cumplirse

- Restricción de atributo
- Restricción de tupla
- Restricción de tabla
- Restricción de base de datos

Procedimental

¿ CÓMO? Indicar cómo debe cumplirse

Estrategias de integridad

Declarativa

¿QUÉ? Indicar qué debe cumplirse

- Restricción de atributo
- Restricción de tupla
- Restricción de tabla
- Restricción de base de datos

Procedimental

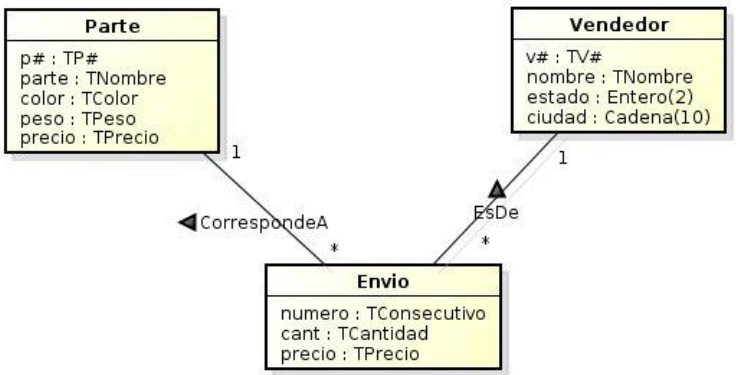
¿ CÓMO? Indicar cómo debe cumplirse

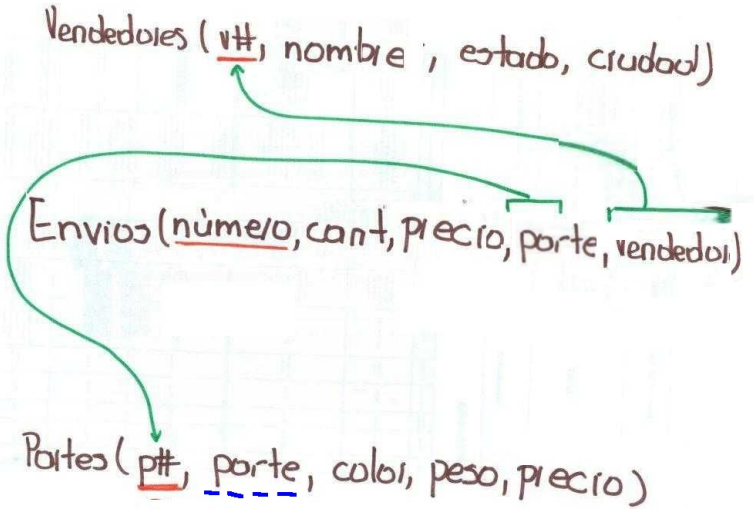
- Acciones de referencia
- Disparadores

Agenda

- Integridad
  - Lo básico
  - Mecanismos
  - Caso: Envíos
- Acciones
  - De base de datos
  - De lenguaje imperativo
  - Excepciones
- Integridad procedimental
  - Acciones referenciales
  - Disparadores
- Caso: Ventas

Envíos


Parte
p# : TP#
parte : TNombre
color : TColor
peso : TPeso
precio : TPrecio
Vendedor
v# : TV#
nombre : TNombre
estado : Entero(2)
ciudad : Cadena(10)
1
CorrespondeA
*
1
EsDe
*
Envio
numero : TConsecutivo
cant : TCantidad
precio : TPrecio


Vendedores (v#, nombre, estado, ciudad)
Envios (número, cant, precio, parte, vendedor)
Partes (p#, parte, color, peso, precio)

Agenda

Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Agenda

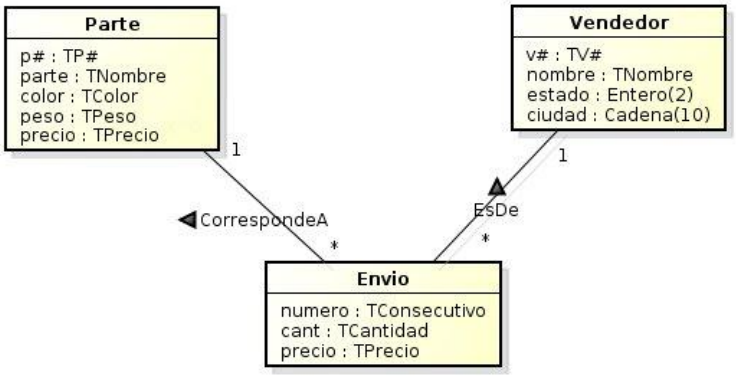
Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

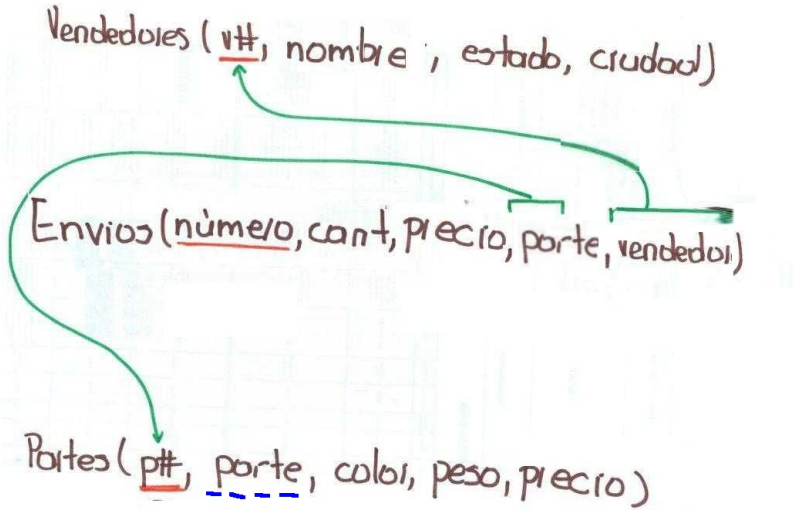
Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Envíos



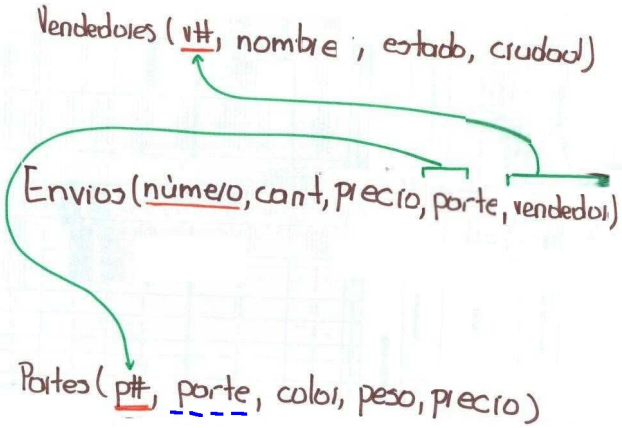


¿Las tres acciones básicas? INSERT DELETE UPDATE

1. `INSERT INTO VENDEDORES(v, nombre, estado, ciudad) VALUES ('V1', 'Smith',20,'Londres');`

   `INSERT INTO VENDEDORES (v, nombre, estado, ciudad) SELECT * FROM OTROS_VENDEDORES;`
2. `DELETE FROM VENDEDORES WHERE v='V2';`
3. `UPDATE VENDEDORES SET estado=2*estado, ciudad='Roma' WHERE ciudad='Paris';`

Envíos



vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

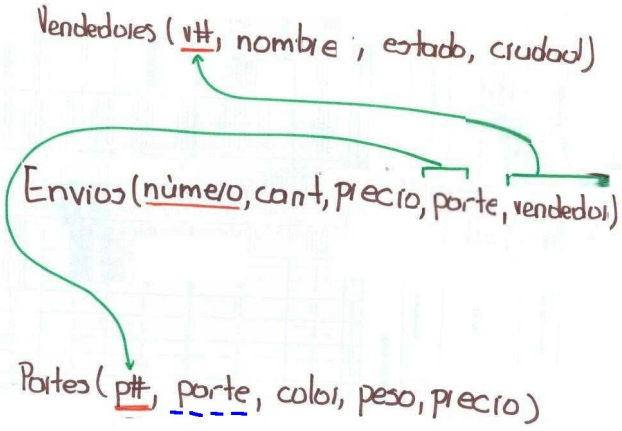
P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

1. Duplicar todas las partes. Código 'C'+p# y el doble del precio

Envíos



vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

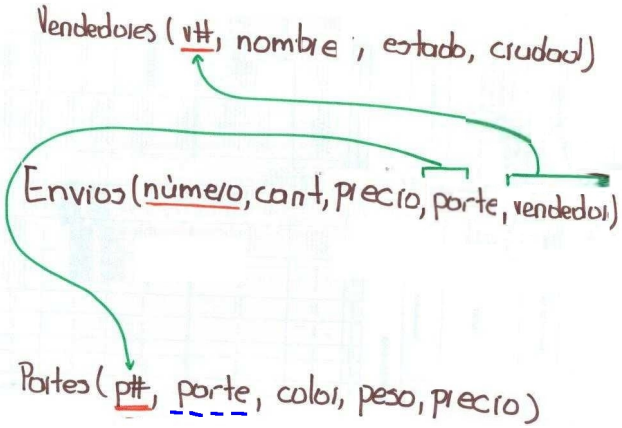
P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

1. Duplicar todas las partes. Código 'C'+p# y el doble del precio
2. Eliminar todas las partes que no tienen envíos con más de 100 unidades

Envíos



vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

1. Duplicar todas las partes. Código 'C'+p# y el doble del precio
2. Eliminar todas las partes que no tienen envíos con más de 100 unidades
3. Incrementar el precio las partes que se han comprado en Roma un 10%

Agenda

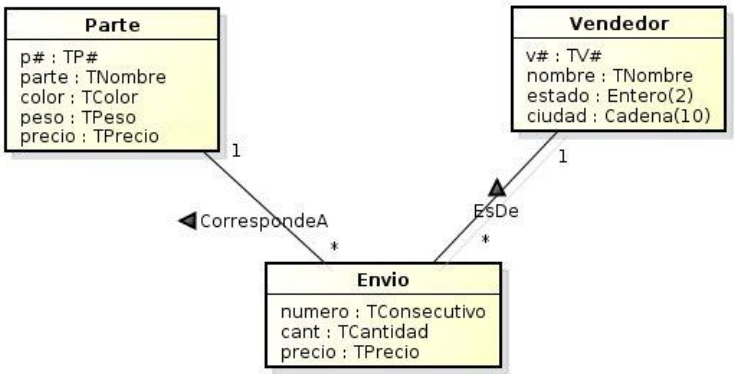
Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

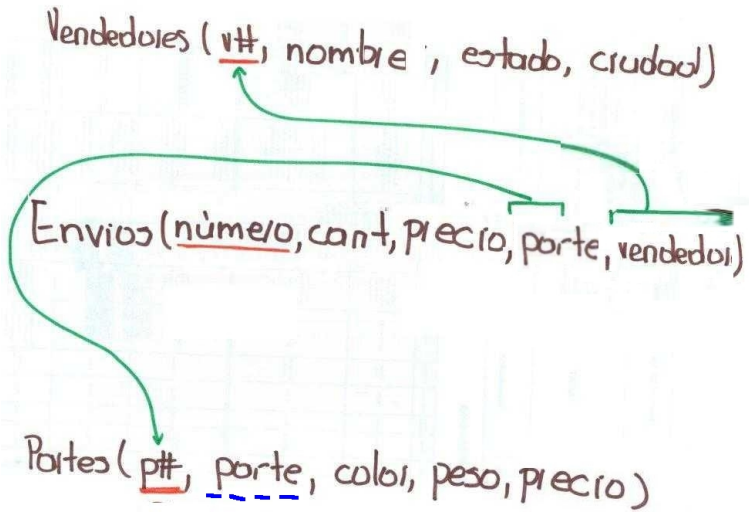
Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

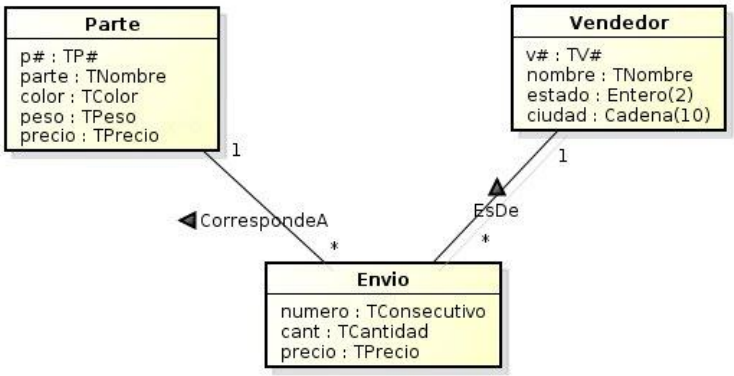
Envíos

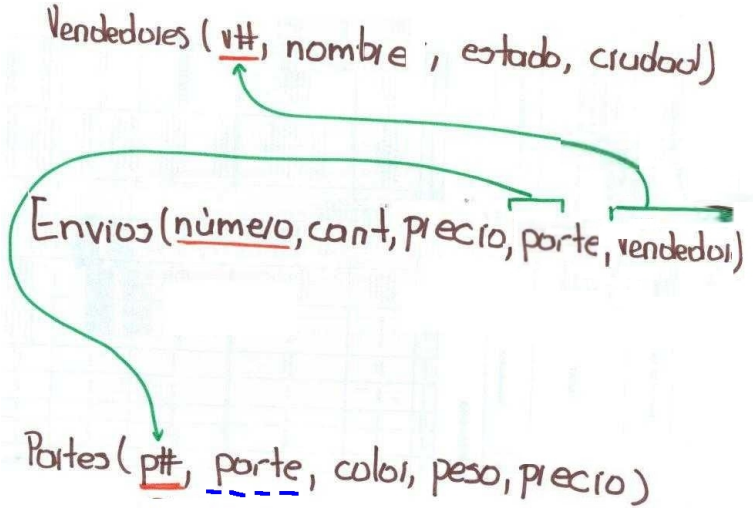




¿Las tres acciones básicas?

Envíos





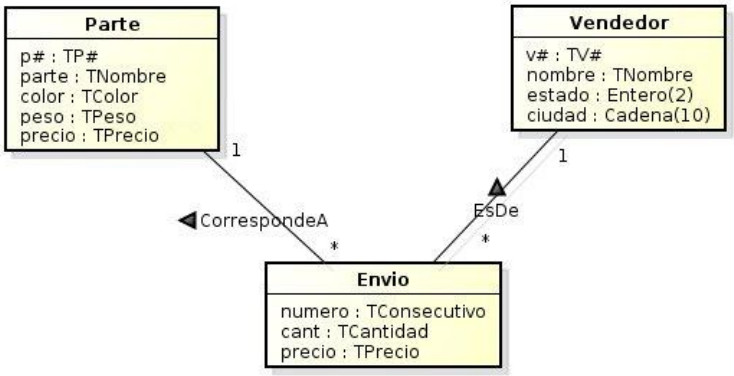
¿Las tres acciones básicas? Asignar Leer Escribir

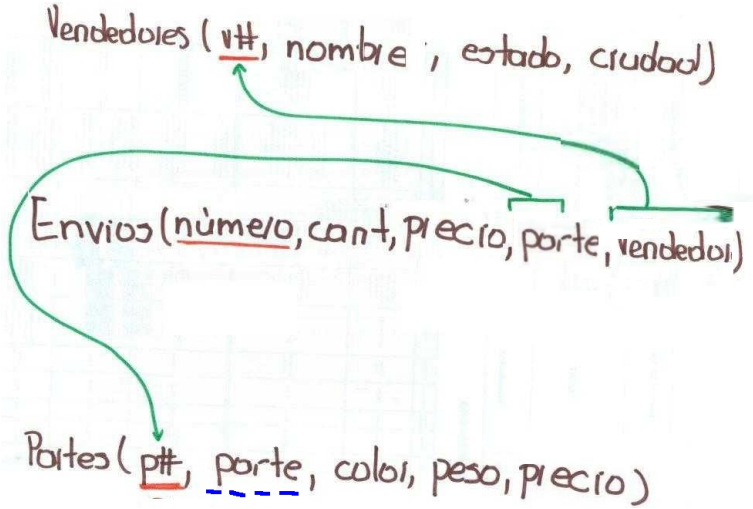
1. `SELECT estado, ciudad INTO _estado, _ciudad FROM vendedores WHERE v_ = 'V1';`

2. `_los_unos CURSOR FOR SELECT estado,ciudad FROM vendedores WHERE estado = 1;`

¿Qué hacen? ¿Errores?

Envíos





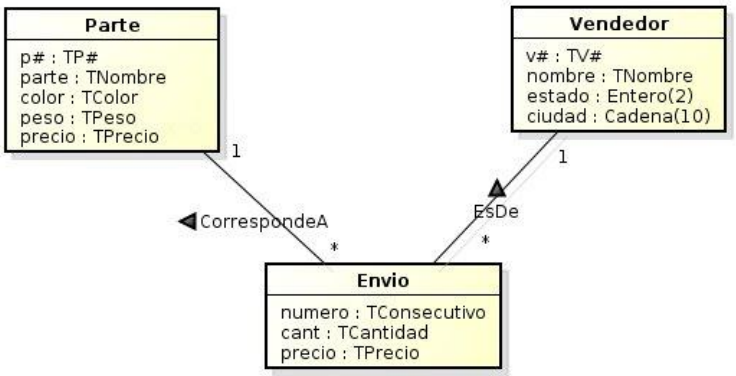
¿Las tres acciones básicas? Asignar Leer Escribir

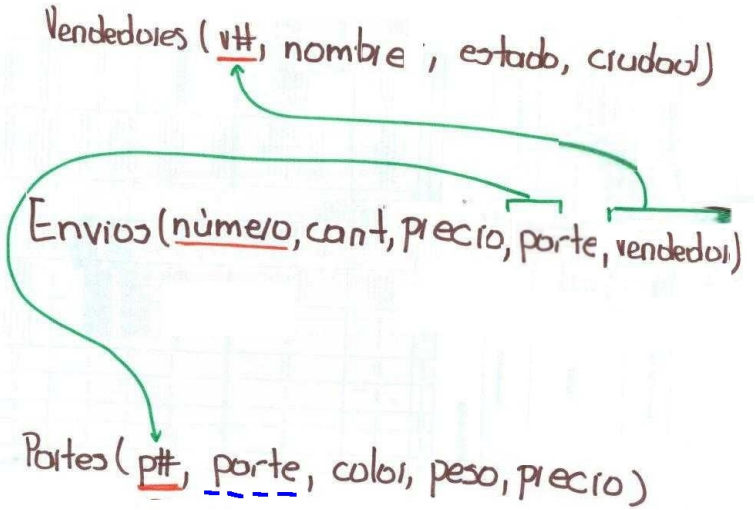
1. `SELECT estado, ciudad INTO _estado, _ciudad FROM vendedores WHERE v_ = 'V1';`

   too_many_rows
2. `_los_unos CURSOR FOR SELECT estado,ciudad FROM vendedores WHERE estado = 1;`

¿Qué hacen? ¿Errores?

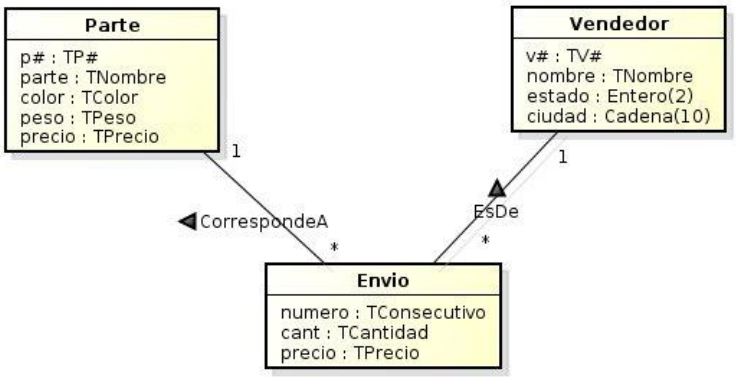
Envíos

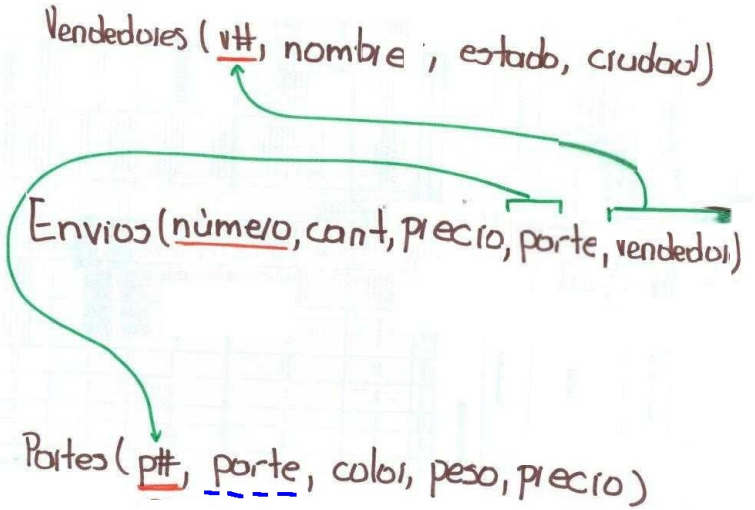




1. Calcular el número a asignar al siguiente envío (_numero)

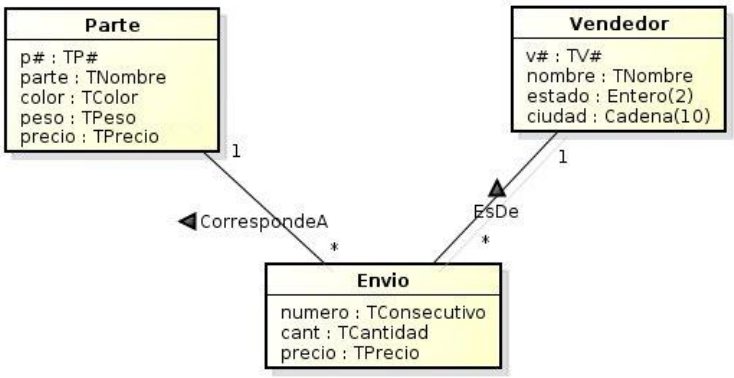
Envíos

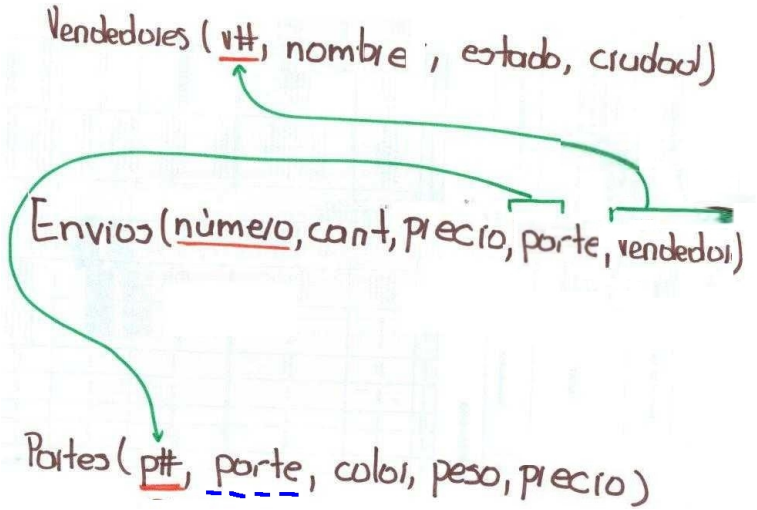




1. Calcular el número a asignar al siguiente envío (_numero)
2. Calcular el precio de una parte (_precio)

Envíos





1. Calcular el número a asignar al siguiente envío (_numero)
2. Calcular el precio de una parte (_precio)
3. Consultar la información del número de envíos de cada ciudad (_por_ciudad)

Agenda

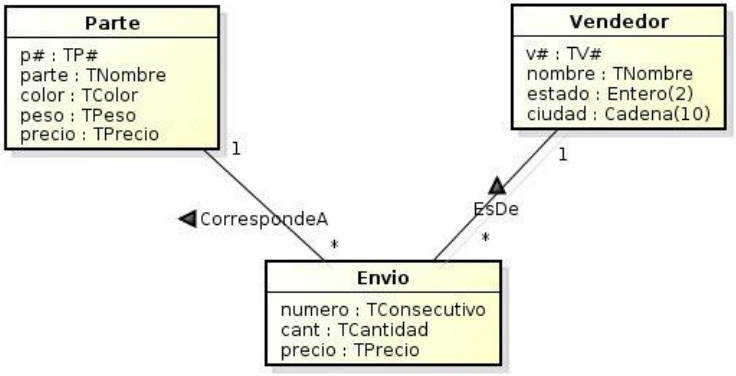
Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

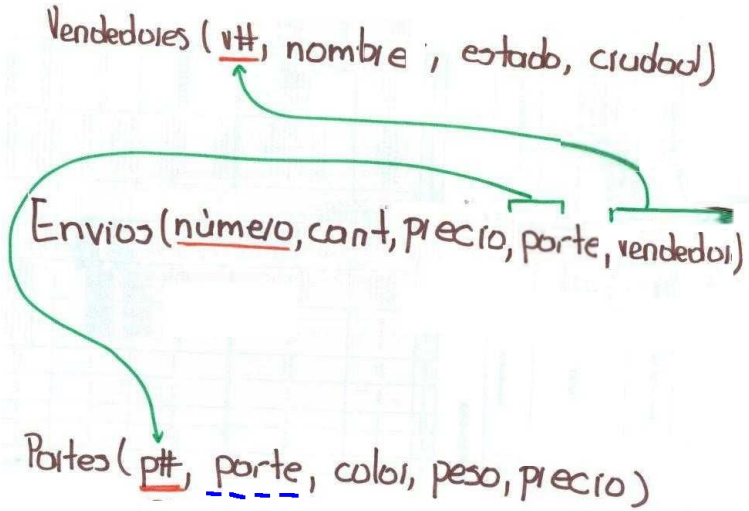
Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Envíos





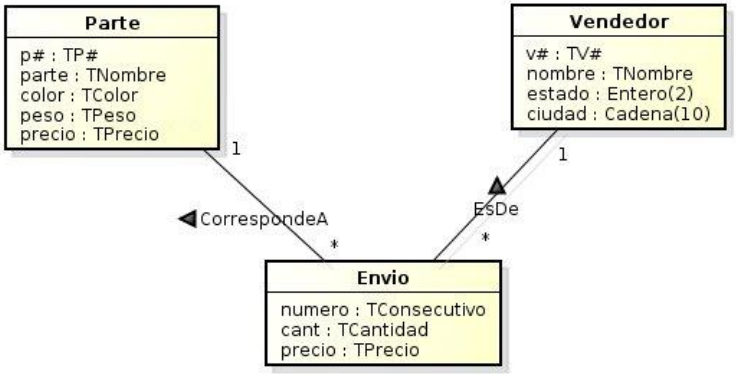
Del sistema

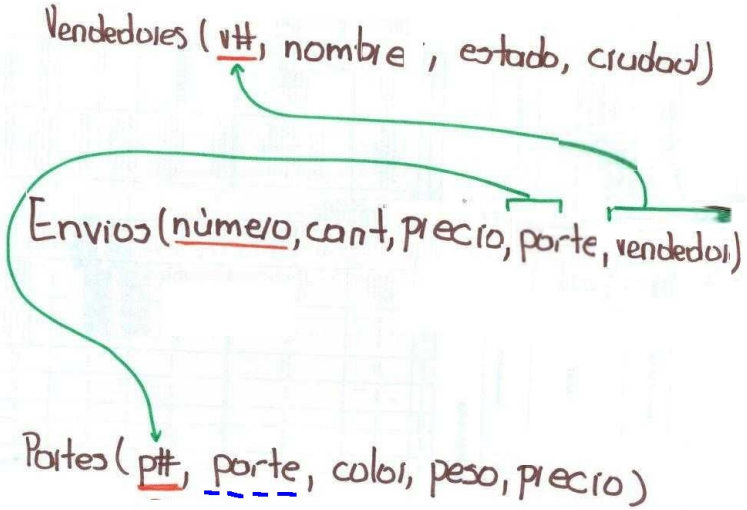
`SELECT INTO`

`too_many_rows (P0003)`

Excepción: mensaje + número

Envíos





Del sistema

`SELECT INTO`

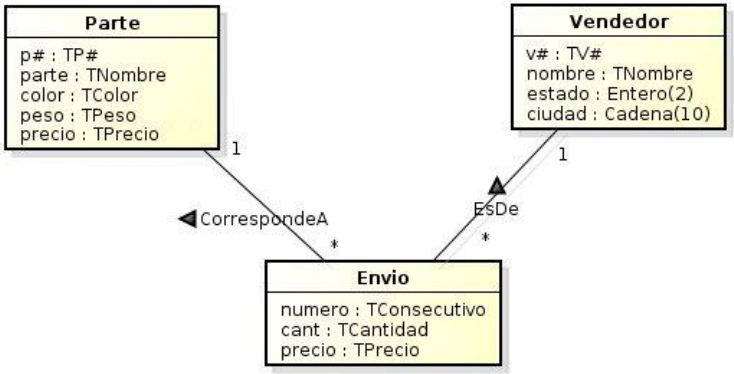
`too_many_rows (P0003)`

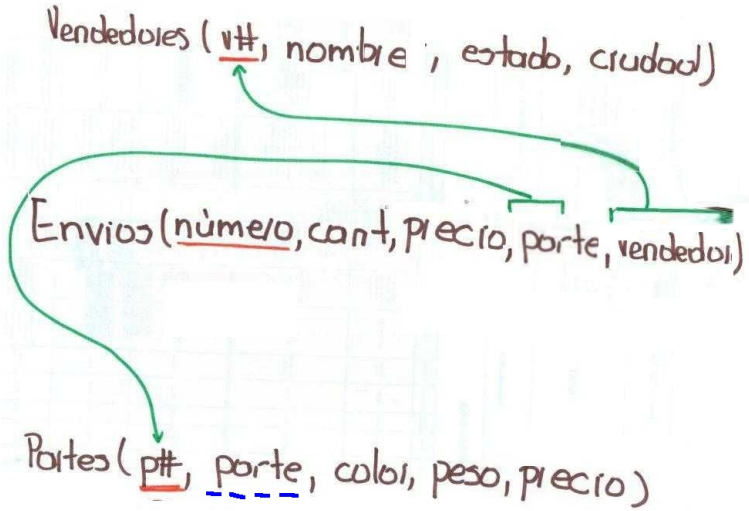
Excepción: mensaje + número

Personalizadas

- `RAISE EXCEPTION USING MESSAGE='Mensaje de error propio', ERRCODE='A0001';`

Envíos





Del sistema

```
SELECT INTO
too_many_rows (P0003)
```

Excepción: mensaje + número

Personalizadas

- `RAISE EXCEPTION USING MESSAGE='Mensaje de error propio', ERRCODE='A0001';`

¿?

1. Lanzar una excepción indicando que no se pueden eliminar los envíos

Agenda

Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Agenda

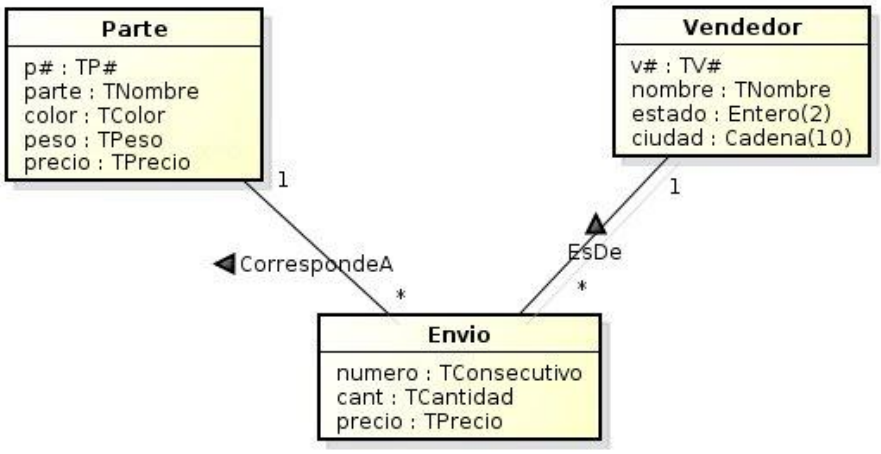
Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

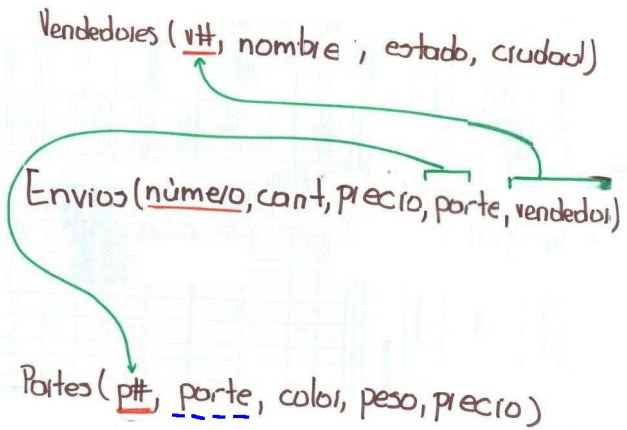
Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Envíos





¿Podemos?

1. Borrar una parte
2. Borrar un vendedor
3. Modificar el código de una parte

Envíos

vendedores

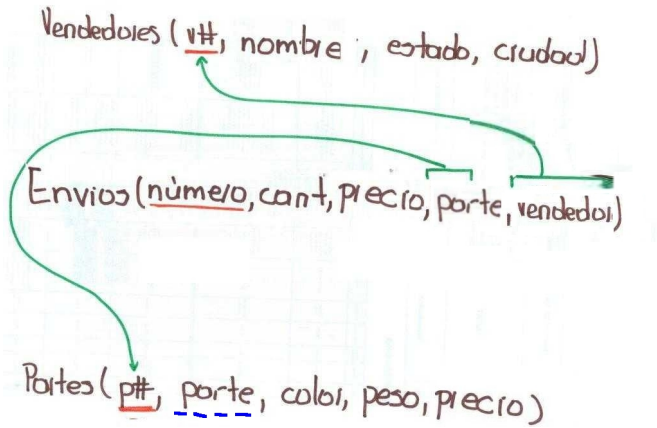
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

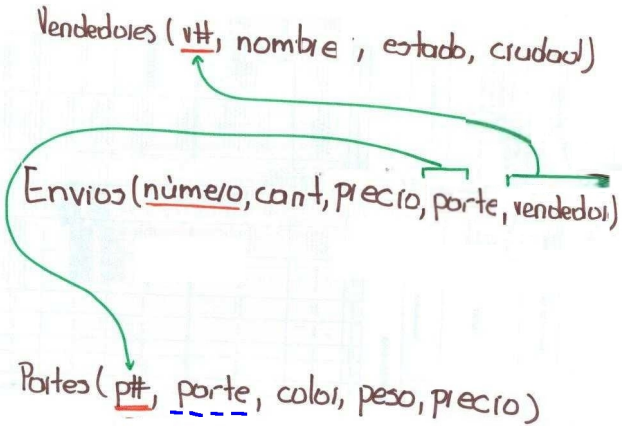


¿Podemos?

1. Borrar una parte ¿Cuáles si?
2. Borrar un vendedor ¿Cuáles si?
3. Modificar el código de una parte ¿Cuáles si?

Actualizar

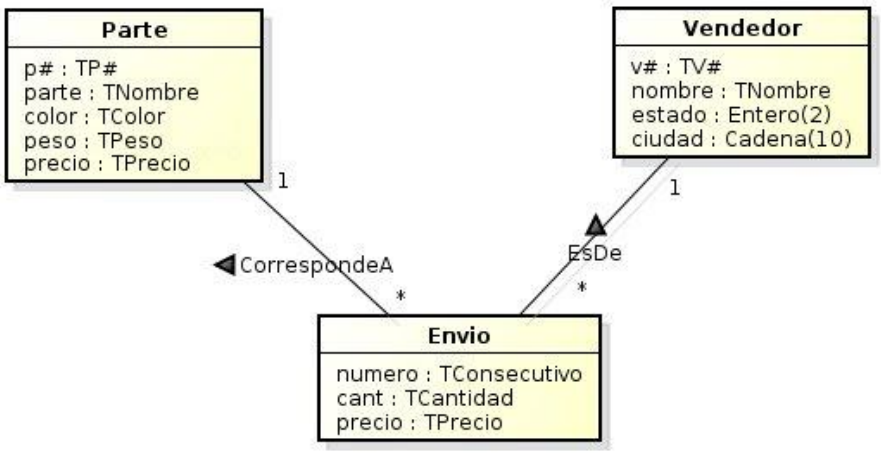
```
CONSTRAINT FK_<nombre_restriccion> FOREIGN KEY (<nombre_atributo>, ...)
    REFERENCES <nombre_tabla> (<nombre_atributo>, ...)
    [ON DELETE (RESTRICT | CASCADE | SET NULL | NO ACTION)]
    [ON UPDATE (RESTRICT | CASCADE | SET NULL | NO ACTION)]
```

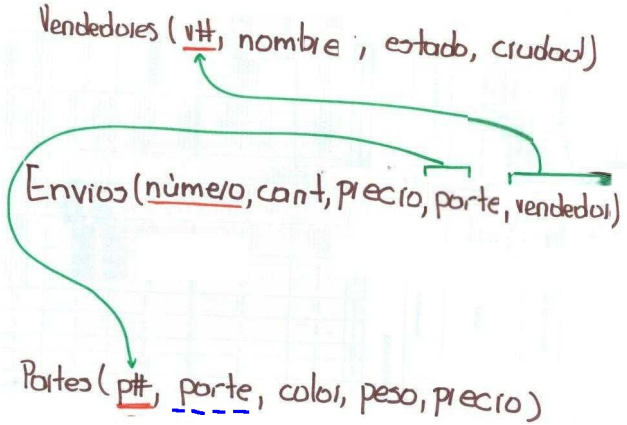


- CASCADE: propaga la acción, si es necesario.
- RESTRICT o NO ACTION PENDIENTE
- SET NULL: deja en NULL, si es necesario

Envíos

```
CONSTRAINT FK_<nombre_restriccion> FOREIGN KEY (<nombre_atributo>, ...)
    REFERENCES <nombre_tabla> (<nombre_atributo>, ...)
    [ON DELETE (RESTRICT | CASCADE | SET NULL | NO ACTION)]
    [ON UPDATE (RESTRICT | CASCADE | SET NULL | NO ACTION)]
```





Propuestas

1. Dejar borrar una parte. Al borrarla se borran sus envíos.
2. Dejar borrar vendedores. Al borrarlo queda en desconocido sus envíos.
3. Dejar modificar el código de una parte. Deja todo con el mismo nuevo código.

Envíos

vendedores

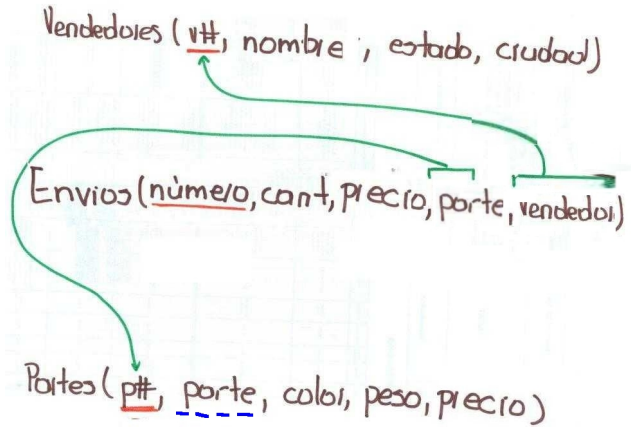
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Propuestas

1. Dejar borrar una parte. Al borrarla se borran sus envíos.
2. Dejar borrar vendedores. Al borrarlo queda en desconocido sus envíos.
3. Dejar modificar el código de una parte. Deja todo con el mismo nuevo código.

`DELETE FROM PARTES WHERE p#='P1'` ¿Qué pasa?

Envíos

vendedores

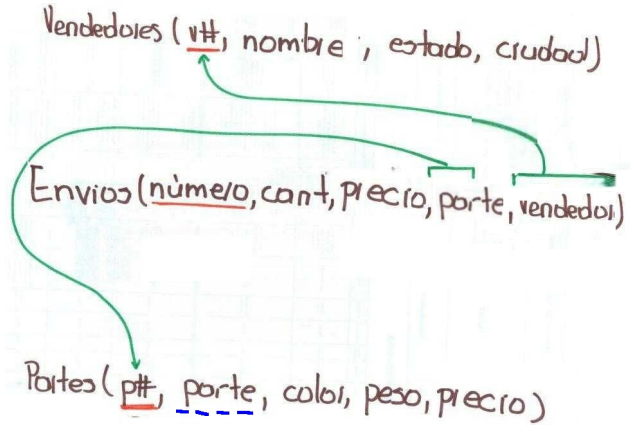
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Propuestas

1. Dejar borrar una parte. Al borrarla se borran sus envíos.
2. Dejar borrar vendedores. Al borrarlo queda en desconocido sus envíos.
3. Dejar modificar el código de una parte. Deja todo con el mismo nuevo código.

`DELETE FROM VENDEDORES WHERE v#='V1'` ¿Qué pasa?

Envíos

vendedores

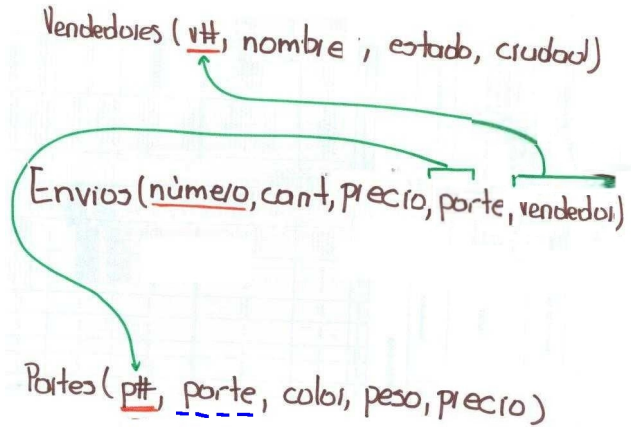
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Propuestas

1. Dejar borrar una parte. Al borrarla se borran sus envíos.
2. Dejar borrar vendedores. Al borrarlo queda en desconocido sus envíos.
3. Dejar modificar el código de una parte. Deja todo con el mismo nuevo código.

`UPDATE PARTES SET p#='P0' WHERE P#='P5'` ¿Qué pasa?

Agenda

Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

Disparadores

SUBPROGRAMA que se ejecuta

en **MOMENTOS** específicos

asociado a **EVENTOS**,

sobre una **TABLA**.

Se ejecuta una o más **VECES**

si se cumple una **CONDICIÓN**

```
CREATE  OR REPLACE TRIGGER nombreDisparador

(BEFORE | INSTEAD OF  | AFTER )

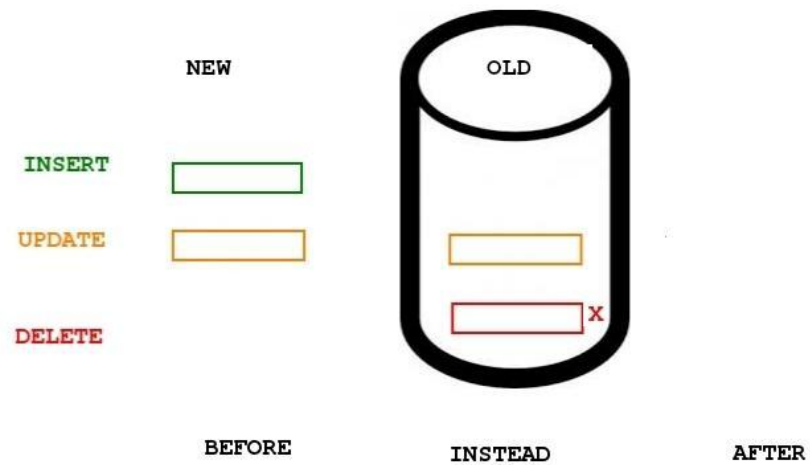
(INSERT|DELETE |UPDATE [OF nombre columnas] )

ON nombre Tabla

[FOR EACH ROW [WHEN (condición)]]

EXECUTE FUNCTION  nombreFuncion() ;
```

[A] significa que puede ir A
(A | B) significa que debe ir A o B
En la tercer línea puede incluirse más de una acción uniéndolas con un OR



Disparadores

SUBPROGRAMA que se ejecuta

`EXECUTE FUNCTION`

en **MOMENTOS** específicos

`BEFORE INSTEAD OF AFTER`

asociado a **EVENTOS**,

`INSERT DELETE UPDATE`

sobre una **TABLA**.

`ON`

Se ejecuta una o más **VECES**

`FOR EACH ROW`

si se cumple una **CONDICIÓN**

`WHEN`

```
CREATE  OR REPLACE TRIGGER nombreDisparador

(BEFORE | INSTEAD OF  | AFTER )

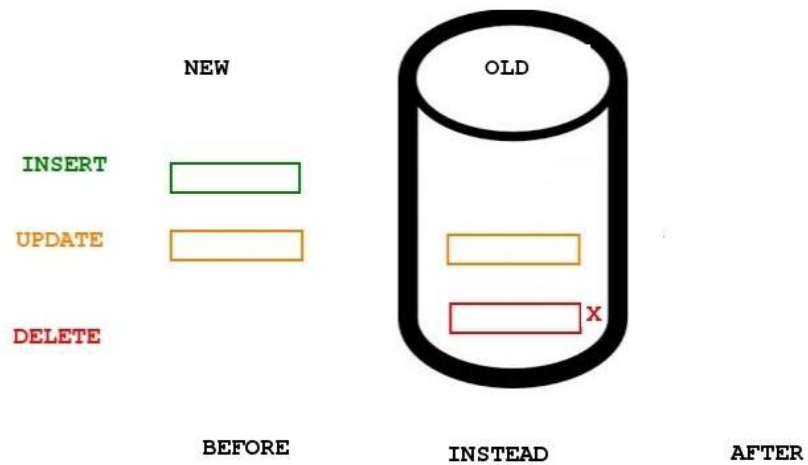
(INSERT|DELETE |UPDATE [OF nombre columnas] )

ON nombre Tabla

[FOR EACH ROW [WHEN (condición)]]

EXECUTE FUNCTION  nombreFuncion() ;
```

[A] significa que puede ir A
(A | B) significa que debe ir A o B
En la tercer línea puede incluirse más de una acción uniéndolas con un OR



Disparadores

El **SUBPROGRAMA** es una función

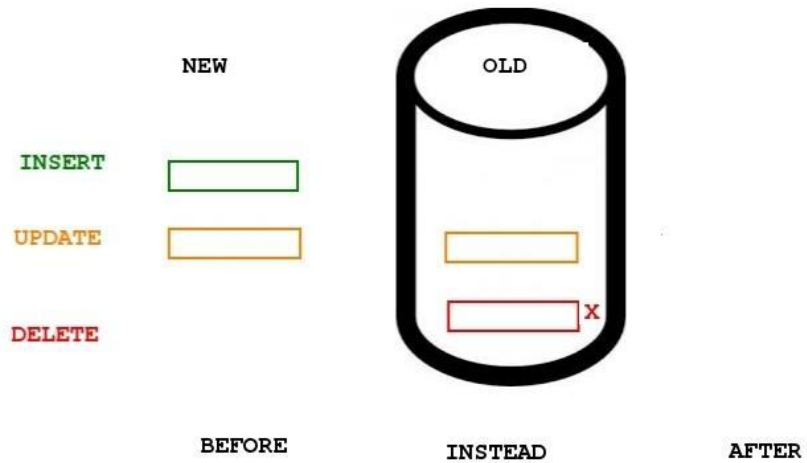
donde se pueden declarar **VARIABLES**

y se indican las **ACCIONES** realizar

el código se escribe en **plpgsql**.

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```



Disparadores

El **SUBPROGRAMA** es una función

```
FUNCTION ...  RETURNS TRIGGER
```

donde se pueden declarar **VARIABLES**

```
DECLARE
```

y se indican las **ACCIONES** realizar

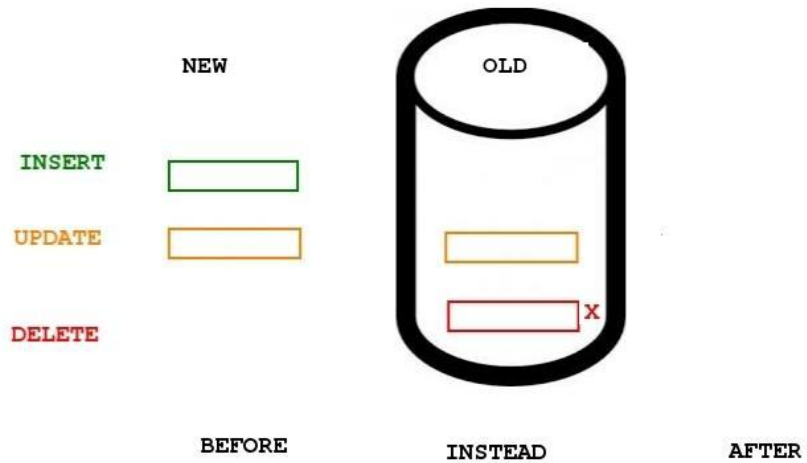
```
BEGIN END
```

el código se escribe en **plpgsql**.

```
$$n ...  $$ LANGUAGE
```

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```



Envíos

```
CREATE OR REPLACE FUNCTION fun_tg_partes_upd_color() RETURNS TRIGGER AS
$$
DECLARE
   veces INTEGER;
BEGIN
   IF OLD.color IN ('amarillo','azul','rojo') THEN
      NEW.color:=OLD.color;
   ELSE
     SELECT COUNT(*) INTO veces FROM partes WHERE color=NEW.color;
     IF (veces >= 30)  THEN
      NEW.color:= 'generico';
      END IF;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER tg_partes_upd_color
  BEFORE UPDATE OF color ON partes
  FOR EACH ROW
EXECUTE FUNCTION fun_tg_partes_upd_color();
```

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
  -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL);
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion();
```

Mo: Actualizar el color de una parte

1. ¿Qué pasa si queremos cambiar el color de una parte rojo a verde? ¿Por qué?

Envíos

```
CREATE OR REPLACE FUNCTION fun_tg_partes_upd_color() RETURNS TRIGGER AS
$$
DECLARE
   veces INTEGER;
BEGIN
   IF OLD.color IN ('amarillo','azul','rojo') THEN
      NEW.color:=OLD.color;
   ELSE
     SELECT COUNT(*) INTO veces FROM partes WHERE color=NEW.color;
     IF (veces >= 30)  THEN
      NEW.color:= 'generico';
      END IF;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER tg_partes_upd_color
  BEFORE UPDATE OF color ON partes
  FOR EACH ROW
EXECUTE FUNCTION fun_tg_partes_upd_color();
```

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
  -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL);
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion();
```

Mo: Actualizar el color de una parte

1. ¿Qué pasa si queremos cambiar el color de una parte rojo a verde? ¿Por qué?
2. ¿Qué pasa si queremos cambiar el color de una parte naranja a verde? Es la primera parte verde. ¿Por qué?

Envíos

```
CREATE OR REPLACE FUNCTION fun_tg_partes_upd_color() RETURNS TRIGGER AS
$$
DECLARE
   veces INTEGER;
BEGIN
   IF OLD.color IN ('amarillo','azul','rojo') THEN
      NEW.color:=OLD.color;
   ELSE
     SELECT COUNT(*) INTO veces FROM partes WHERE color=NEW.color;
     IF (veces >= 30)  THEN
      NEW.color:= 'generico';
      END IF;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER tg_partes_upd_color
  BEFORE UPDATE OF color ON partes
  FOR EACH ROW
EXECUTE FUNCTION fun_tg_partes_upd_color();
```

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
  -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL);
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion();
```

Mo: Actualizar el color de una parte

1. ¿Qué pasa si queremos cambiar el color de una parte rojo a verde? ¿Por qué?
2. ¿Qué pasa si queremos cambiar el color de una parte naranja a verde? Es la primera parte verde. ¿Por qué?
3. ¿Qué pasa si queremos cambiar el color de una parte rosada a verde? Ya hay treinta verdes. ¿Por qué?

Envíos

vendedores

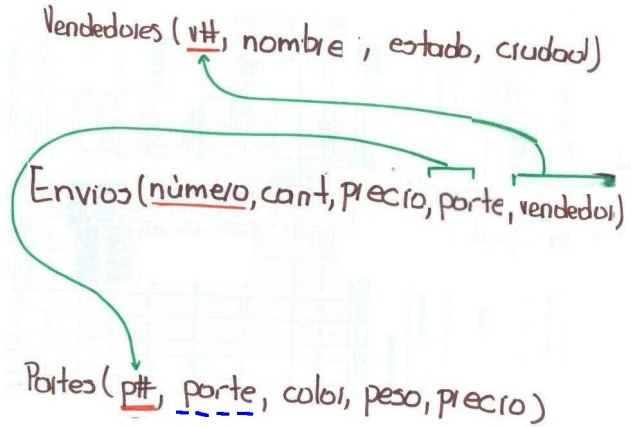
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Ad: Adicionar un envío

1. ¿Instrucción actual para adicionar un nuevo envío de 25 partes de P1 del proveedor V1?

Envíos

vendedores

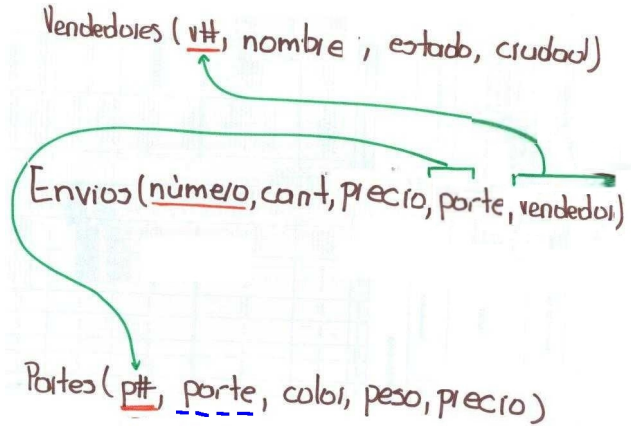
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Ad: Adicionar un envío

1. ¿Instrucción actual para adicionar un nuevo envío de 25 partes de P1 del proveedor V1?
2. ¿Instrucción deseada con datos mínimos? ¿Qué puede calcular el sistema?

Envíos

vendedores

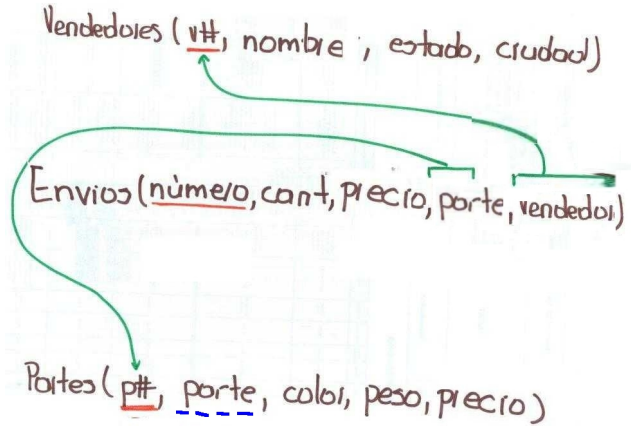
v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200



Ad: Adicionar un envío

1. ¿Instrucción actual para adicionar un nuevo envío de 25 partes de P1 del proveedor V1?
2. ¿Instrucción deseada con datos mínimos? ¿Qué puede calcular el sistema?
3. ¿Cómo lograrlo?

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Ad: Adicionar un envío

1. Disparador `tg_envios_ins` : Adicionar un envío con datos mínimos

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Ad: Adicionar un envío

1. Disparador `tg_envios_ins` : Adicionar un envío con datos mínimos
2. Seguimiento a adicionar un nuevo envío de 25 partes de P1 del proveedor V1

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Mo: Actualizar el precio de una parte, no deben bajar

1. Instrucción para subir el precio de todas las partes a 100

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Mo: Actualizar el precio de una parte, no deben bajar

1. Instrucción para subir el precio de todas las partes a 100
2. ¿Qué pasa? ¿Qué desearíamos que pasara?

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Mo: Actualizar el precio de una parte, no deben bajar

1. Instrucción para subir el precio de todas las partes a 100
2. ¿Qué pasa? ¿Qué desearíamos que pasara?
3. Disparador `tg_partes_upd_precio` : Modificar el precio de una parte

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

Mo: Actualizar el precio de una parte, no deben bajar

1. Instrucción para subir el precio de todas las partes a 100
2. ¿Qué pasa? ¿Qué desearíamos que pasara?
3. Disparador `tg_partes_upd_precio` : Modificar el precio de una parte
4. Seguimiento

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

El: No se deben poder eliminar los envíos

1. Instrucción para eliminar los envíos de 'P4'

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

El: No se deben poder eliminar los envíos

1. Instrucción para eliminar los envíos de 'P4'
2. ¿Qué pasa? ¿Qué desearíamos que pasara?

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

El: No se deben poder eliminar los envíos

1. Instrucción para eliminar los envíos de 'P4'
2. ¿Qué pasa? ¿Qué desearíamos que pasara?
3. Disparador `tg_envios_del` : Eliminar un envío

Envíos

vendedores

v_	nombre	estado	ciudad
V1	Smith	2	Londres
V2	Jones	1	París
V3	Blake	3	París
V4	Clark		Londres
V5	Adams	3	Atenas

partes

P_	parte	color	peso	precio
P1	tuerca	rojo	12	20
P2	perno	verde	17	110
P3	tornillo	azul	17	60
P4	tornillo	rojo	14	40
P5	leva	azul	12	200
P6	engrane	rojo	19	56

envios

numero	vendedor	parte	cant	precio
1	V1	P1	300	20
2	V1	P2	200	100
3	V1	P3	400	40
4	V1	P4	200	40
5	V1	P5	100	160
6	V2	P1	300	20
7	V2	P2	400	110
8	V3	P2	200	110
9	V4	P3	200	60
10	V4	P4	300	40
11	V4	P5	400	200

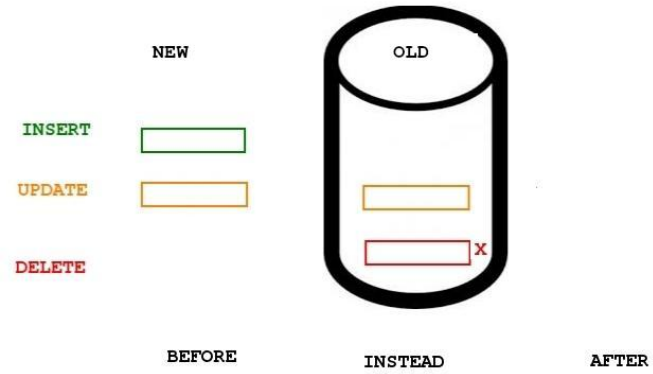
```
CREATE OR REPLACE FUNCTION  nombreFuncion() RETURNS TRIGGER AS
$$
DECLARE
   -- Definición de variables
BEGIN
  -- Instrucciones del subprograma
  RETURN (NEW| OLD | NULL) ;
END;
$$ LANGUAGE plpgsql

CREATE  OR REPLACE TRIGGER nombreDisparador
(BEFORE | INSTEAD OF  | AFTER )
(INSERT|DELETE |UPDATE [OF nombre columnas] )
ON nombre Tabla
[FOR EACH ROW [WHEN (condición)]]
EXECUTE FUNCTION   nombreFuncion() ;
```

El: No se deben poder eliminar los envíos

1. Instrucción para eliminar los envíos de 'P4'
2. ¿Qué pasa? ¿Qué desearíamos que pasara?
3. Disparador `tg_envios_del` : Eliminar un envío
4. Seguimiento

Disparadores

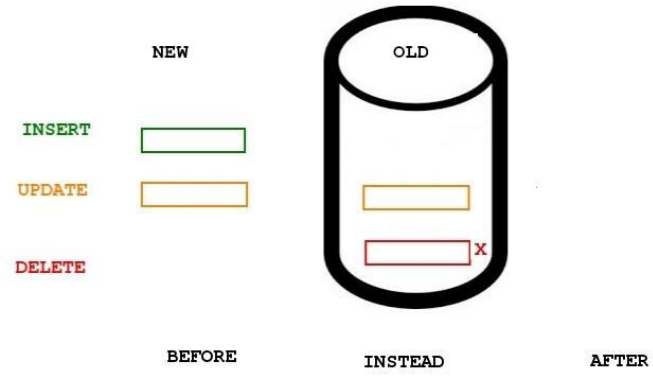


¿Para qué?

1. Implementar reglas de integridad

2. Mover lógica de aplicación a datos

Disparadores



¿Para qué?

1. Implementar reglas de integridad
   - Evitando acciones no deseadas
   - Automatizando acciones deseadas
2. Mover lógica de aplicación a datos
   - Centralizando acciones
   - Aumentando la eficiencia

Agenda

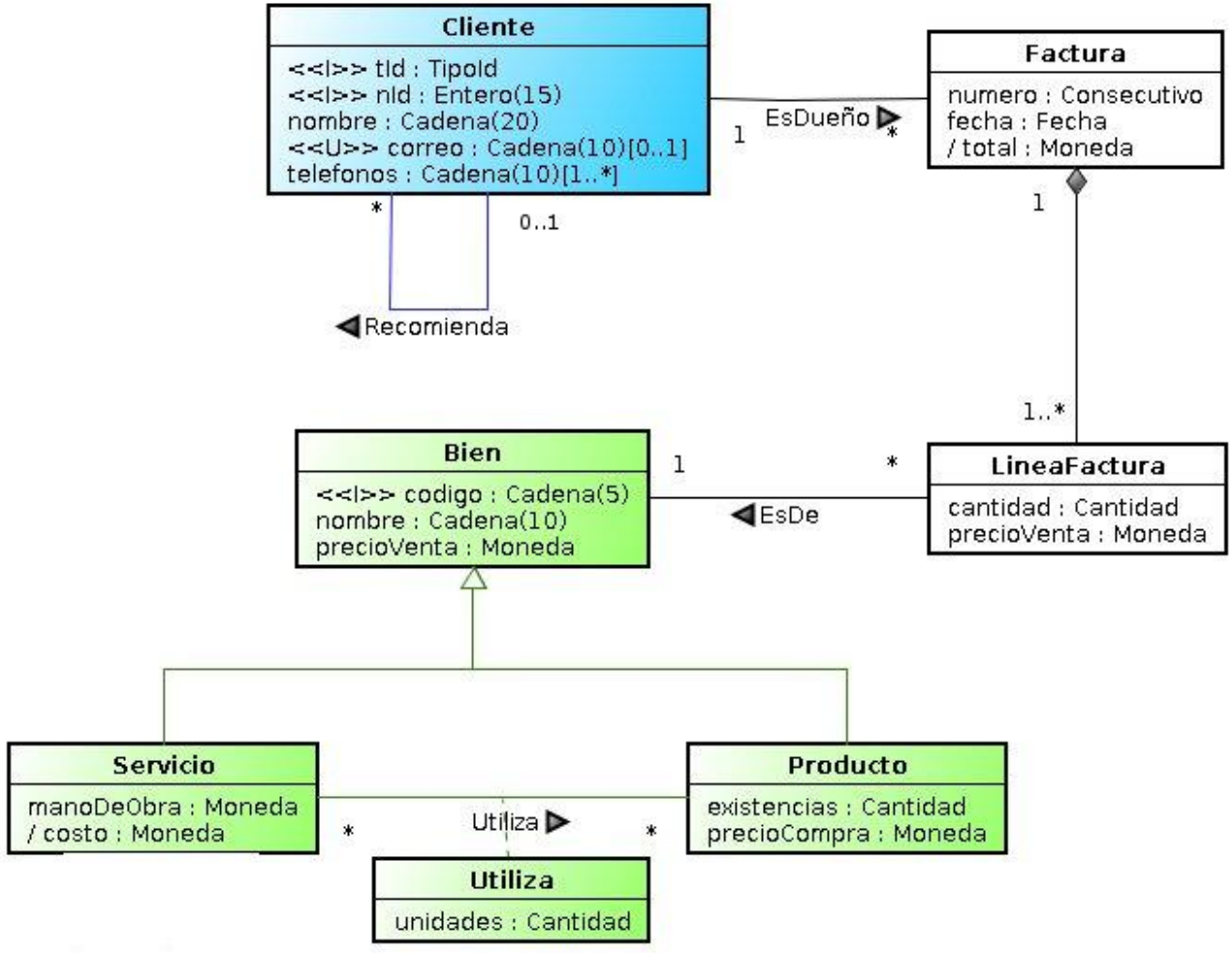
Integridad
- Lo básico
- Mecanismos
- Caso: Envíos

Acciones
- De base de datos
- De lenguaje imperativo
- Excepciones

Integridad procedimental
- Acciones referenciales
- Disparadores

Caso: Ventas

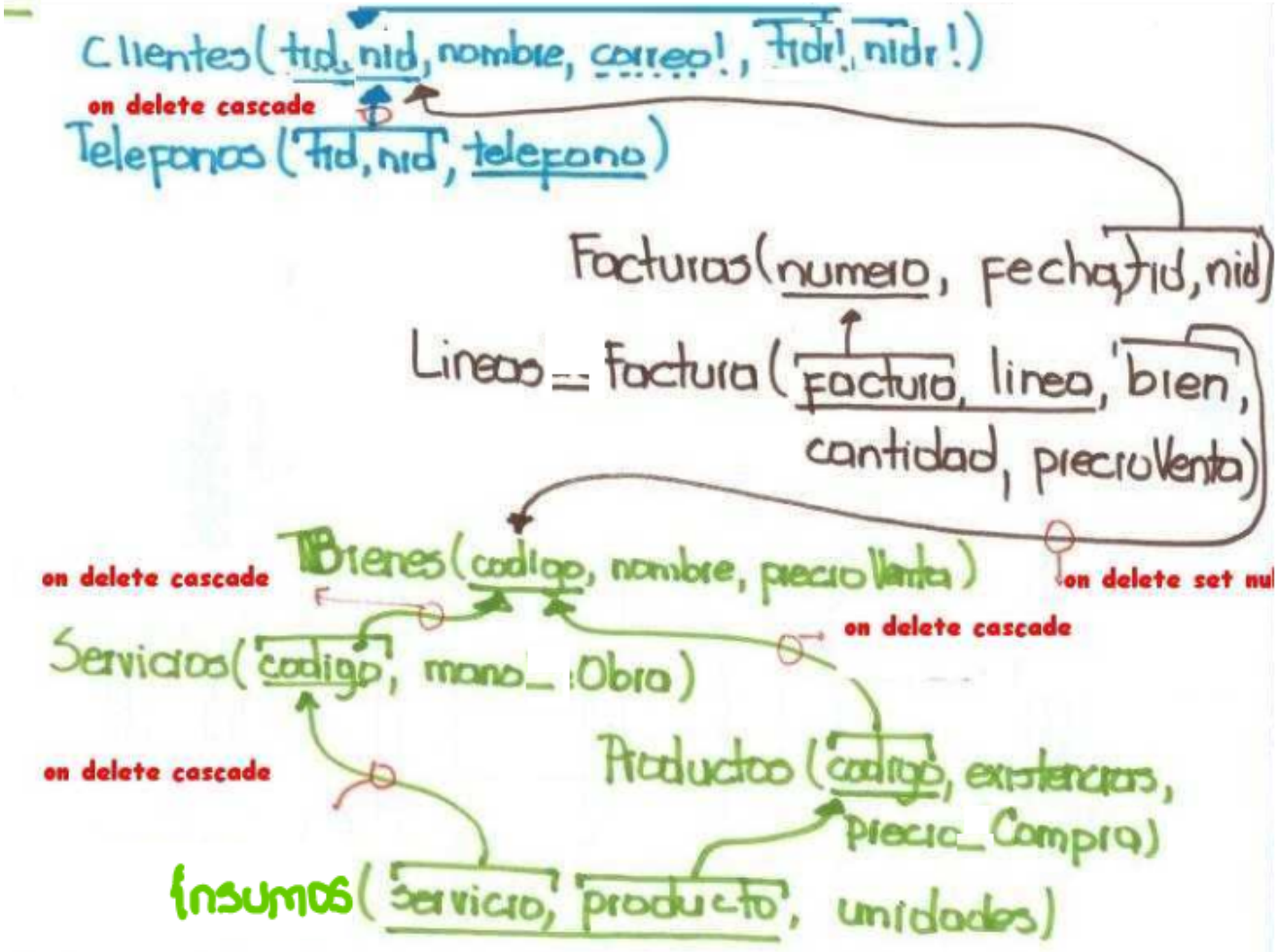
Ventas. Conceptual



Al adicionar una línea de factura se debe:

- Almacenar el precio de venta actual del bien
- Disminuar las existencias de los productos

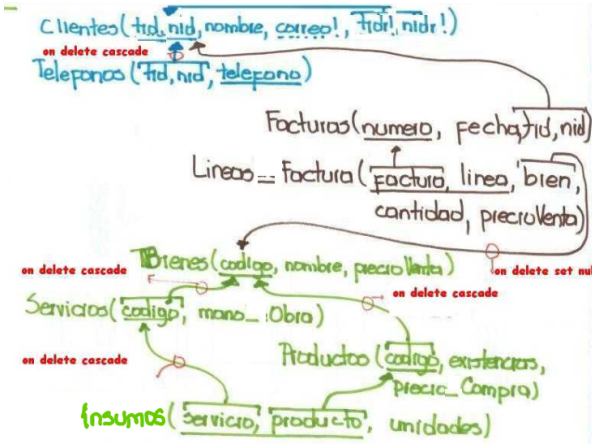
Ventas. Relacional.



- ¿Disparador asociado a cuál tabla?
- ¿Qué tipo de disparador? (Evento, momento, veces)

Ventas. Codigo. Disparador

Ventas




```
CREATE OR REPLACE FUNCTION fun_tg_lineas_factura_ins() RETURNS TRIGGER AS
$$
DECLARE
   insumo RECORD;
BEGIN
   SELECT COUNT(*)+1 INTO NEW.linea FROM lineas_factura
      WHERE factura=NEW.factura;
   SELECT precio_venta INTO NEW.precio_venta FROM bienes
      WHERE codigo=NEW.bien;
   --Si es un producto
   UPDATE productos SET existencias=existencias - NEW.cantidad
      WHERE codigo= NEW.bien;
   --Si es un servicio
   FOR insumo IN SELECT * FROM insumos WHERE servicio=NEW.bien LOOP
      UPDATE productos
         SET existencias=existencias-insumo.unidades*NEW.cantidad
      WHERE codigo=insumo.producto;
   END LOOP;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER tg_lineas_factura_ins
BEFORE INSERT ON lineas_factura
FOR EACH ROW
EXECUTE FUNCTION fun_tg_lineas_factura_ins();
```